

Chapter #5

Conceptual Modeling

Challenges of designing data marts:

- Entity-relationship model is widely used for database design, but it is not well suited for multidimensional data, which is common in data marts (data mart is based on multidimensional view of data).
- The ERM expressive enough to represent the majority of concepts necessary for data modeling.
- However, the basic ERM is not able to accurately highlight the distinctive features of the multi-dimensional modeling.
- That's why its use for data mart is not suitable.

Challenges of designing data marts using star schema:

Designers, focus on the structure and relationships of the data within the data mart without considering the underlying

physical implementation.

→ For this, they directly define the star schema structure.

Star Schema:-

- Specific type of data model used in data marts.

- Characterized by a central fact table surrounded by dimension tables.

- A star schema is a relational schema.

→ Directly using star schemas for conceptual design is similar to designing a relational database without first creating an ER-schema, which can lead to problems with responsiveness to requirements, maintenance and reuse.

→ In case of data warehouse, star schemas can be particularly problematic because:

1. They are almost completely denormalized.

2. Do not properly code the functional dependencies that define hierarchies - means design and implementation of the data mart do

not accurately represent the relationships and dependencies between different levels of data.

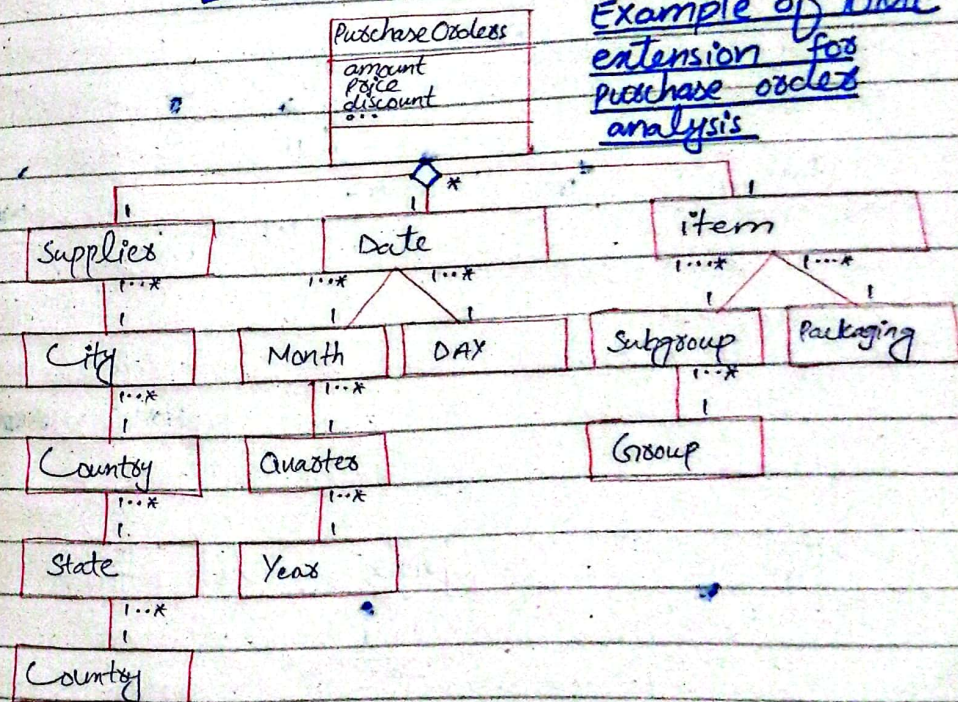
→ Combining the strength of ERM and star schemas to create effective data mart design.

→ There are different approaches to multidimensional modeling some of which based on:

1. ERM extensions

2. UML extensions

Example of UML extension for purchase order analysis



Non-usable attributes:

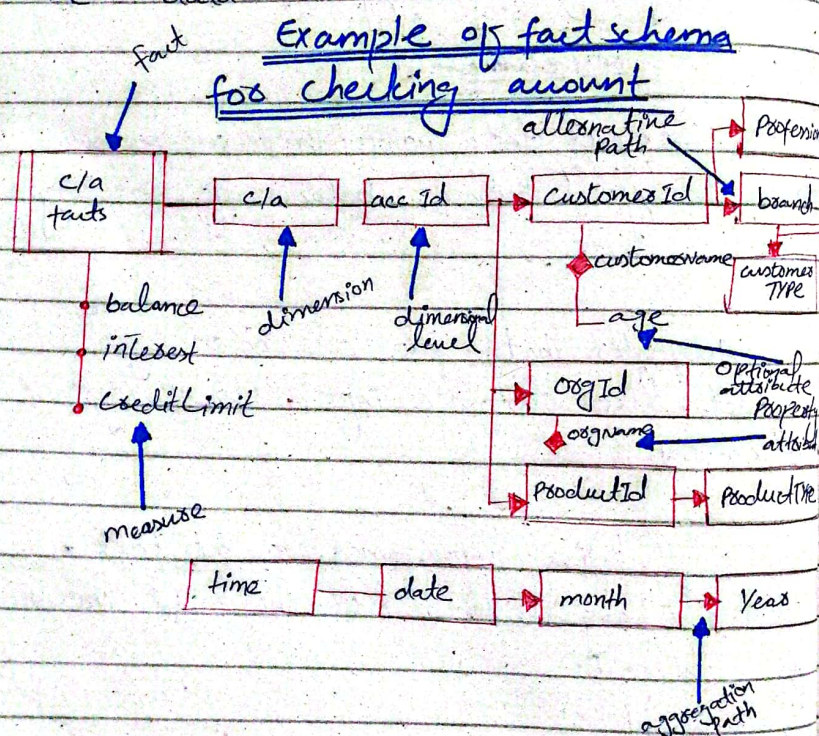
Attributes that are not used for aggregation but may be useful for other purposes.

Optional attributes:

Attributes that may or may not be present for given fact.

Alternate aggregation paths:

Different ways to group and analyze the data.



Dimensional fact Model

• is a conceptual model created specifically to function as data mart design support.

• Way to picture and understand data in multi-dimensional way.

Goal of DFM:

(i) Clear design:

Give effective support to Conceptual design.

(ii) Easy queries:

Users can ask questions about the data in a simple way.

(iii) Better Communication:

Designers and users can communicate more easily about requirement specification.

(iv) Stable platform:

Build a stable platform for logic design.

(v) Clear and expressive design documentation:

Provide clear and expressive design documentation.

- A fact schema is a diagram that shows the relationships between dimensions and measure in a data warehouse.

Fact - schemata:

⇒ The conceptual representation generated by the DFM consists of a set of fact schemata.

⇒ Fact schemata model:

- Facts
- Measures
- Dimensions
- Hierarchies

Fact:

⇒ is a concept relevant to decision making, processes.

⇒ like a story or event that can track.

⇒ it's something that happens and can be measured.

⇒ They can change over time, however some of the concepts represented in the database are completely static.

Examples

⇒ Sales:

When a product is sold.

⇒ Purchases:

When someone buys something.

⇒ Complaints:

When a customer is not happy.

example how facts are dynamic:

- ⇒ The number of sales can go up or down.
- ⇒ A customer complaint can be solved.

Measures:

⇒ are the numbers that describe the facts.

⇒ Numerical property of fact.

⇒ Describes quantitative fact aspect that is relevant to analysis.

Examples

⇒ Quantity:

How many products were sold.

⇒ Total sales

Each sale is measured by the number of units sold, the unit price and the total receipts.

Note:

- Most measures are numbers, however, sometimes a fact has no measure. In this case, only occurrence of an event is recorded.

⇒ if fact has no measure then the fact schema is said to be empty.

Dimensions:

⇒ are different ways to look at data.

⇒ are lenses through which we see data.

⇒ it is a fact property with a finite domain.

⇒ Also describes an analysis coordinate of the fact.

⇒ A fact has more than one dimensions.

Examples:

⇒ Time:

Year, month, day, hour, minute.

⇒ Product:

Category, brand, size, color.

⇒ Sale:

Product stores and dates.

Note:

⇒ Connection between measures and dimensions is expressed at the extensional (data level) - rather than schema level.

Primary Event:

⇒ is a single instance or particular occurrence of a fact.

⇒ Like a snapshot of the fact at a particular point in time.

⇒ Identified by one n-ple made up of a value for each dimension.

n-ple: The unique combination of values is called "n-ple."

⇒ A value for each measure is associated with each primary event.

⇒ Dimensions are normally used to identify and select primary events.

Example of primary event:

o Sale of 10 packages of detergent:

This is a specific instance of a sale. It happened on a particular day (10/10/2008) at a specific store (SmartMart) and involved a specific product (Shiny Detergent).

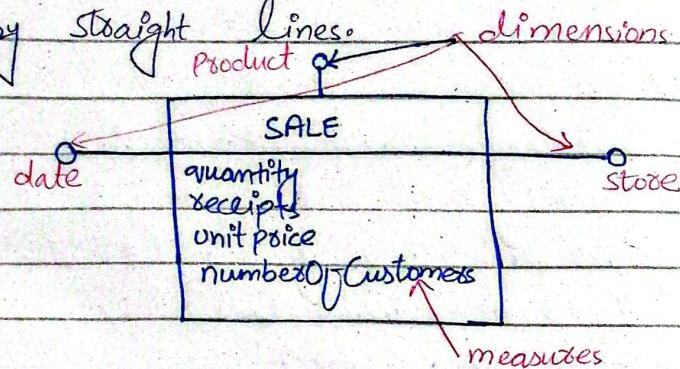
o The dimensions are time (10/10/2008), product (Shiny Detergent), store (SmartMart) and quantity (10 packages).

o The measure is total sales (\$25).

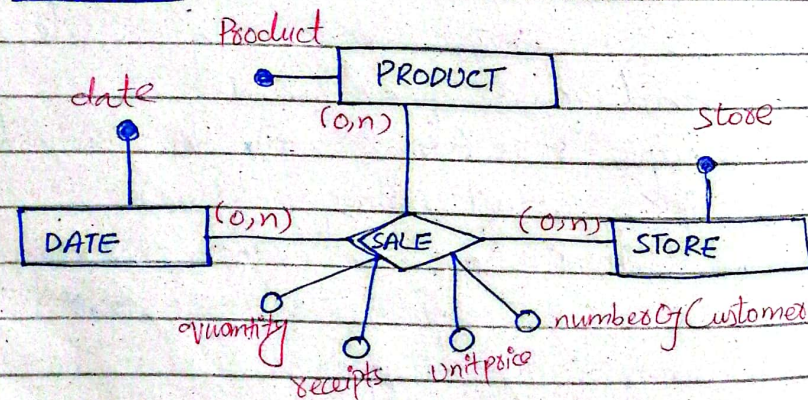
Simple fact Schema for sales

⇒ A fact is represented by a box that displays the fact name along with the measure names.

⇒ Small circles represent the dimensions, which are linked to the fact by straight lines.



ER-schema corresponding to fact Schema.



⇒ A fact expresses a many-to-many association between dimensions.

Due to this, the ER schema corresponding to a fact schema consists of mainly n-ary relationship

That models the fact, among entities that model dimensions

The measures are attributes of this relationship.

Dimensional attribute:

stands for dimensions and other possible attributes, always with discrete values, that describe them.

Hierarchies are used to represent relationships between different dimensional attributes. Dimensional attributes are represented for example: by circle.

A product is described by its type, by the category to which it belongs, by its brand, and by the department in which it sold. Then product, type, category, brand and department will be dimensional attributes.

Hierarchy:

- A hierarchy is a directed tree whose nodes are dimensional attributes and whose edges represent connections or model many-to-one association between dimensional attribute pairs.
- It includes a dimension, placed at the tree root and all of the dimensional attributes describe it.

for example:

A product hierarchy might have a "product category" at the top, with different "product types" branching off from it. This shows that product types belong to specific categories.

Note:

- Hierarchy term is different in ER-modeling and multidimensional modeling.
- In ER-modeling, hierarchies are used to represent inheritance (IS-A) relationships between entities.
- In multidimensional modeling, hierarchies are created in data warehouses using axes to

represent relationships between attributes.

- These axes can be used to express different types of relationships, such as aggregation, generalization and association.

(some point on next page)

Secondary Events:

⇒ are group of primary events that share common characteristics.

⇒ Given a set of dimensional attributes (usually from different hierarchies), each n-tuple of their values identifies a secondary event.

⇒ Each secondary event has a value for each measure, which sums up the values of that measure in the corresponding primary events.

Example:

Sales can be grouped according to product category, month and store city.

The n-tuple (StoreCity: Miami, product: Shiny, month: 10/2008) identifies a secondary event that aggregates all sales of the Shiny product in Miami Stores during October 2008.

Note 2:

⇒ All attributes and measures within a fact schema must have distinct names.

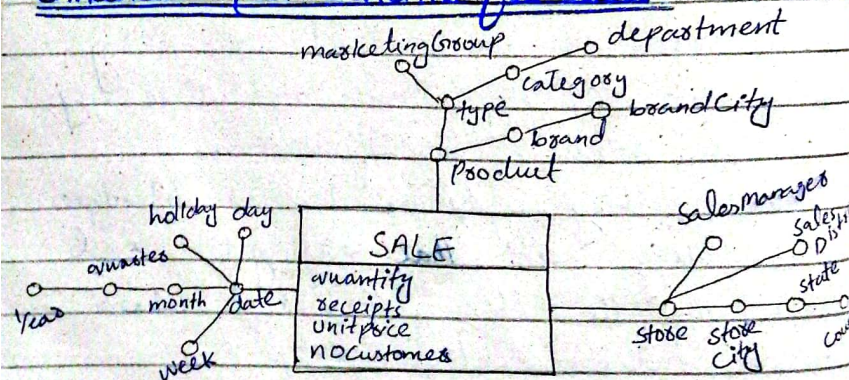
⇒ The convention proposed by Kimball et al. (1998) suggests building names from three components:

1. Object
2. Classification
3. Qualifiers

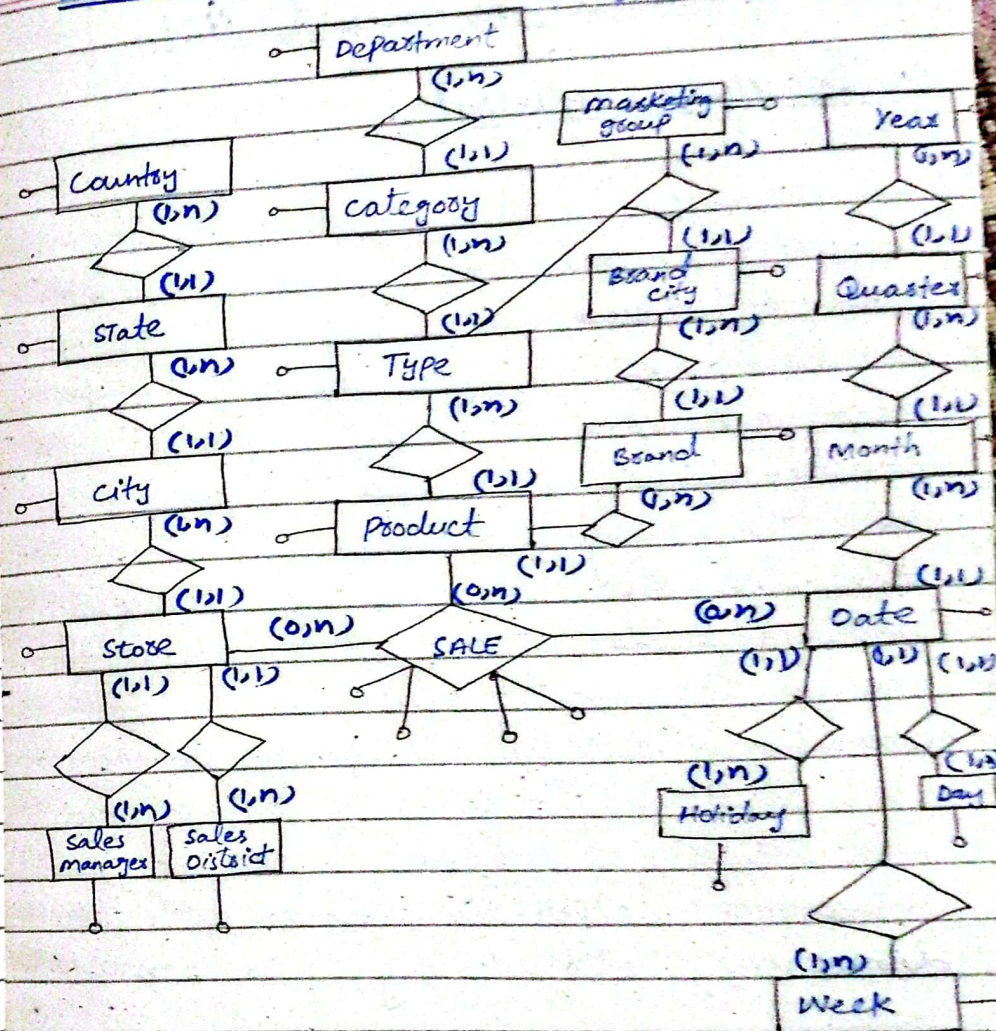
⇒ Functional dependencies establish many-to-one associations between the values of a dimension.

e.g. A product belongs to one product type, but a product type can have many products.

Enhanced fact schema for Sales:



Ex - schema corresponding to fact schema:



Advanced Modeling

1. Descriptive Attributes:

⇒ Attributes that provide additional information about a dimensional attribute in a hierarchy.

⇒ This information is not typically used for aggregation or sorting.

⇒ They are always at the "leaves" of hierarchy.

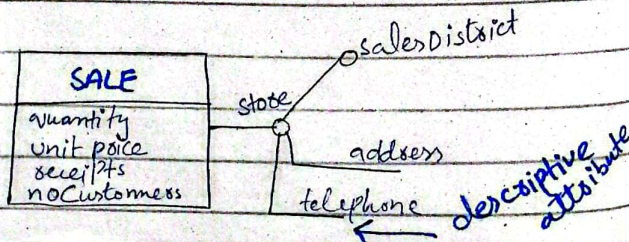
⇒ They are represented by horizontal lines in the DEM and as attributes of entities in the ERM.

example: (Dimensional attributes are store managers)

○ Store address ○ Manager's phone

○ Product weight

⇒ Descriptive attributes often are tied to dimensional attributes by 1-to-1 association.



2. Cross-Dimensional attribute:

⇒ These attributes are dimensional or descriptive attributes whose values are defined by the combination of two or dimensional attributes.

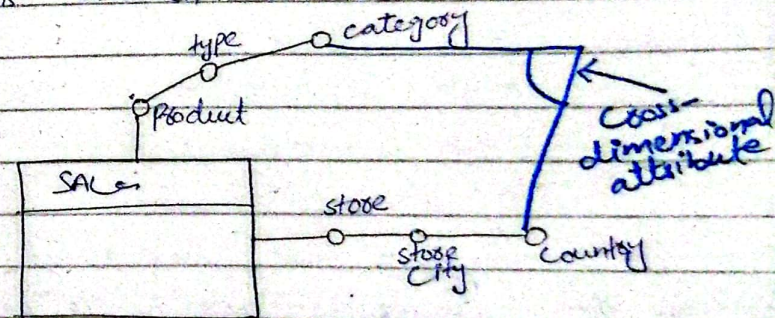
⇒ The value of a cross dimensional attribute depends on the values of other attributes from different hierarchies.

example:

Product VAT (value added Tax), which might depend on both the product category and the country where it is sold.

Representation: They are represented by joining the axes that define the attribute with a circular arc in the fact schema.

In ERM, they are represented by associations between entities.



3. Convergence:

⇒ Convergence deals with the structure of hierarchies in a fact schema.

⇒ Hierarchies may not always be simple trees, as two or more paths might connect the same dimensional attributes.

⇒ This happens when there are functional dependencies between attributes, even if they don't belong to the same hierarchy.

example:

- A store hierarchy where stores are grouped into cities, states and countries, but also into sales districts.

- if store's country can be determined either through:

Store → storecity → state → country

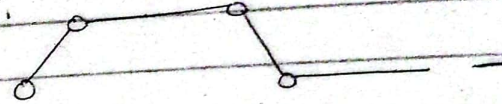
or Store → salesDistrict → country

there is a convergence.

Convergence are marked by arrows on the converging axes in the fact schema.

⇒ In ERM, they are represented by redundant associations between entities.

⇒ Similar looking attribute don't always indicate a convergence. They might have different meanings and need to be represented separately.



4. Shared Hierarchies:

⇒ Shared hierarchies are portions of hierarchies that are replicated two or more times within a fact schema.

⇒ These are hierarchies that are used in multiple parts of a fact schema.

example:

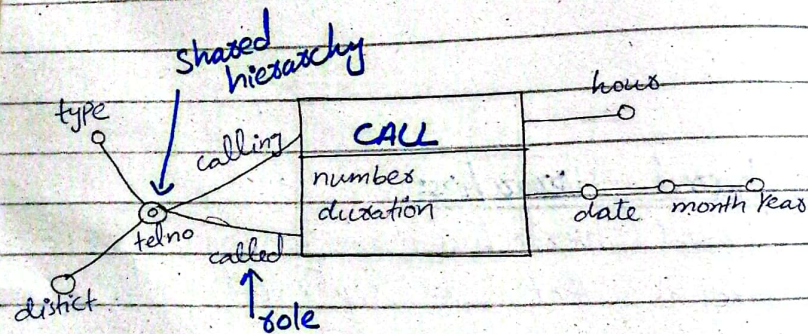
- Temporal hierarchies (e.g., date, month, year)

- Geographical hierarchies (city, state, country etc.)

⇒ if have sharing hierarchies, maintain distinct names for all attributes to avoid ambiguity.

⇒ Graphic notation used for this is use double circles to represent shared attributes and their dependents. This is more concise representation.

Diagram:



5. Multiple Axes:

⇒ Used to represent many-to-many relationship between attributes.

⇒ A single value of one attribute can be associated with multiple attribute values and vice versa.

example:

⇒ Book sales fact schema where a book can have multiple authors and an author can write multiple books.

Representation:

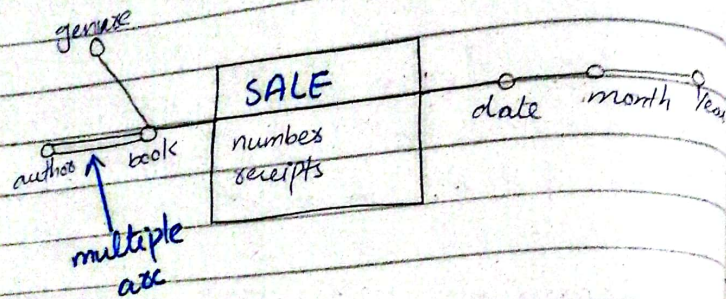
⇒ Represented by drawing two axes between the attributes.

Coefficients:

⇒ A coefficient should be assigned to each multiple axis to define the aggregation rule for the m-2-M relationship. This ensures that aggregations are calculated correctly along hierarchies that include multiple axes.

⇒ This is important for modeling where one entity can be associated with multiple entities of other types.

Diagram:



6. Optional Assoc:

⇒ These are used to model situations where certain attributes or dimensions might not always be defined for all events.

⇒ An optional assoc is marked with a dash (-) in the diagram.

Two cases to consider:

An optional assoc determines a dimensional attribute.

⇒ An optional assoc determines a dimension
Coverage of optional assoc:

⇒ When two or more optional ascs exit from the same attribute, coverage should specify.

⇒ Coverage defines the relationship between the different options involved.

⇒ These are four types of coverage:

Total (T):

- At least one of the children is linked to each value of the parent attribute.

Disjoint (D):

- A value of the parent attribute is linked to at most one of the children.

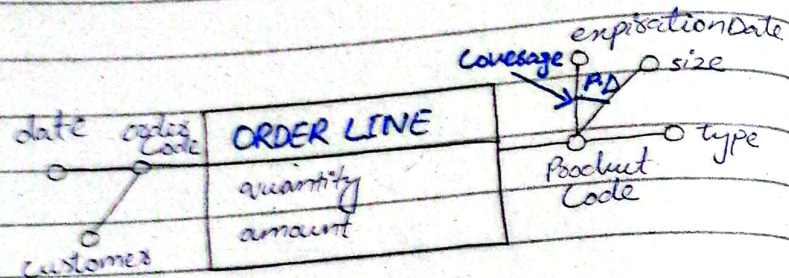
Partial (P):

- Values of the parent attribute exist for which all of the children are undefined.

Overlapping (O):

- Values of the parent attribute exist that link to values of two or more children.

Diagram:



7. Incomplete Hierarchies:

⇒ Hierarchies where one or more levels of aggregation are missing for some instances.

⇒ All entities in the hierarchy have complete information for all levels.

ex:

⇒ Geographically hierarchy where some cities might be missing their country or state information.

Representation:

⇒ By marking the missing attributes with a dash (-).

The term "en-many" is sometimes used to describe incomplete hierarchies.

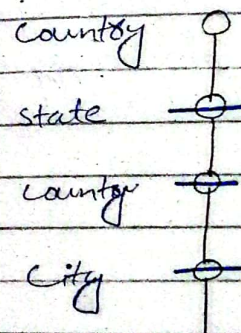
Optional Arc VS incomplete hierarchy

○ Entire attributes or dimensions are missing

○ Some values within a hierarchy level are missing.

⇒ Modeling incomplete hierarchies can be done using either optional arc or by explicitly representing the missing levels.

Incomplete geographical hierarchy diagram:



8. Recursive Hierarchies:

⇒ Useful for modeling self-referential relationships.

⇒ In these hierarchies parent-child relationships among levels are consistent, but the instances can have different length.

example:

Company organization chart where employees report to managers and managers themselves are employees.

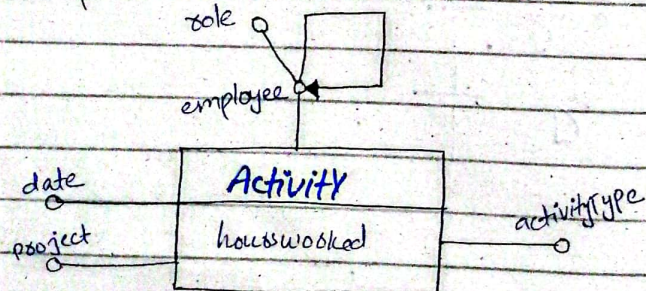
Representation:

⇒ Represented by loop.

Caution:

⇒ Not always possible to distinguish the different levels of aggregation semantically at the fact schema level.

example:



9. Additivity:

⇒ Refers to how measures can be aggregated along different dimensions.

⇒ measure can be classified into three types:

1. flow measures:

o These measures are additive along all dimensions.

example:

Sales quantity & no of units

Sold.

2. Level Measures:

o These measures are not additive along temporal dimensions but are additive along non-temporal dimensions.

example:

inventory levels and hierarchy.

3. Unit price Measures:

o These measures are not additive along any dimension.

example:

Unit price & Discount percentage.

valid aggregation Operators

	Temporal	Non-temporal
flow measures	SUM, AVG, MIN, MAX	SUM, AVG, MIN, MAX
Level measures	AVG, MIN, MAX	SUM, AVG, MIN, MAX
Unit measures	AVG, MIN, MAX	AVG, MIN, MAX

Events and Aggregation

Events:

⇒ Database house are designed to store and analyze large amount of data. To efficiently manage and analyze this data, it's broken down into smaller units called events.

Primary Events

are the most detailed level of data, representing the smallest unit of activity.

ex: a single sale of a product.

Secondary Events

Derived from primary events through aggregation.

ex: Represent higher level of detail, like total sales for a specific product category.

○ Events are building blocks of data warehouse.

○ Cube represent all possible combinations of ^{primary} events, but some cubes are sparse.

Missing Events:

They have different interpretations depending on the type of fact schema.

1. Lossy-grained fact schema:

In this type of schema, missing events are assumed to have

relevant values, typically zero.

ex:

if a product was not sold in a store on a certain day, it's assumed that the sales quantity was zero.

2. Lossless-grained fact schema:

In this schema, missing events truly represent the absence of an event.

Representing missing events:

Two approaches to represent missing events in lossy-grained schemas:

1. Coverage schema:

⇒ A new schema is created explicitly represent the absence of an event.

2. Null values in the original schema:

⇒ use null values in the original schema to represent missing events.
○ Lead to wasted space if the number of missing events is high.

Aggregation:

⇒ The process of combining data from multiple rows into a single row, typically by summing up values.

Group-by sets:

⇒ A subset of a dimensional attributes in a fact schema that does not contain two attributes related by a functional dependency.

⇒ Group-by sets define different levels of aggregation.

Primary group-by sets

○ The set of all dimensions in a fact schema.

Secondary group-by sets

○ Subsets of the primary group-by set.

◦ Represents the most detailed level of aggregation.

◦ Represent different levels of aggregation.

ex:

$G_1 = \{ \text{Product, state, quarters} \}$

$G_2 = \{ \text{Country, date} \}$

$G_3 = \{ \text{Year} \}$

$G_4 = \{ \}$

Roll-up orders:

⇒ Defines the hierarchy of group-by sets.

⇒ Specifies how secondary group-by sets can be rolled up to the primary group-by set.

Relationship:

A group-by set G_1 is said to roll-up to another group-by set G_2 ($G_1 < G_2$) if for each attribute in G_1 , there is a corresponding attribute in G_2 belonging to the same hierarchy.

EX^B

The group-by set (type, store, month) rolls up to the group-by set (type, store, quarters) because the date hierarchy is rolled up to the same hierarchy.

Lattice Structure:

⇒ The set of all possible group-by sets in a fact schema form a lattice structure.

⇒ The primary group-by set is the maximum element in the lattice.

⇒ The empty group-by set is the minimum element in the lattice.

⇒ For each pair of group-by sets, there exists a superior group-by set (roll-up) and an inferior group-by set (roll-down).

Aggregating additive measures

⇒ Additive measures are those that can be summed up to calculate the total value for higher level of aggregation.

ex: The total sales for a year can be calculated by summing up the sales for each month.

Associative property:

⇒ The associative property of the sum operator allows for efficient calculation of aggregated values at different levels.

Aggregating Non-additive Measures:-

⇒ Cannot simply summed up. They require different techniques:

1. Distributive

2. Algebraic

3. Holistic

1. Distributive Measures:

⇒ These measures can be calculated by applying the same aggregation function to partial aggregates.

ex:

• MIN & MAX functions are distributive.

• Minimum value of a group

can be found by finding the value of each subgroup and then take minimum of those minimums.

2. Algebraic Measures:

⇒ These measures require additional information beyond the partial aggregates to calculate the correct values.

ex:

AVG function is algebraic. Cannot simply the average the averages of subgroups to get the overall average. Find Count and sum of each subgroup to calculate the correct average.

3. Holistic Measures:

⇒ These measures cannot be calculated from partial aggregates at all. They require access to the underlying data.

Aggregating With Convergence & Cross-Dimensional Attributes

Convergence:

⇒ Convergence refers to a situation where a lower-level dimension

is related to higher level dimension through a clear hierarchy.

⇒ Aggregation of convergence is straightforward.

⇒ Roll-up values from lower levels to higher levels without ambiguity.

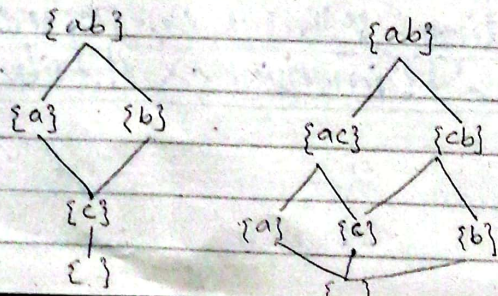
Cross-dimensional attributes

⇒ Cross-dimensional attributes are attributes that are related to multiple dimensions.

ex: a product might be associated with both a category and a brand.

⇒ Aggregation with cross-dimensional attributes can be more complex.

⇒ Ensure that the aggregation is done correctly based on the relationships between the dimensions and the cross-dimensional attribute.



Aggregating with optional or multiple Arcs

Optional Arcs

⇒ An optional arc exists between two dimensions when there is a one-to-many relationship. This means that one value in the higher-level dimension can be associated with multiple values in the lower-level dimensions.

⇒ To handle optional arc during aggregation, introduce a fictitious "No value" to represent missing values.

⇒ This allows to roll-up values from lower levels to higher levels, even there are missing values.

Multiple Arcs

⇒ Multiple Arcs occur when a lower-level dimension is related to multiple higher-level dimensions.

⇒ Aggregation of "there is neg" is complex. Assigning weights to arc can help in allocating values to different dimensions.

Aggregation in empty Fact-Schemas:

- ⇒ Schemas that don't have any measures.
- ⇒ In these schemas, the primary events only record the occurrence of events in an application domain.
- ⇒ In empty fact schemas, the only aggregation possible is counting the number of primary events. This is achieved using the COUNT aggregation operator.

Aggregating with functional Dependencies among dimensions:

- ⇒ A functional dependency exists when one dimension determines the value of another dimension.

Aggregating with incomplete or Recursive Hierarchies:

Incomplete Hierarchies:

- ⇒ Incomplete hierarchies occur when there are missing values for one or more attributes in the hierarchy.
- ⇒ Lead to difficulties in aggregation, as it becomes challenging to roll up values to higher levels of the hierarchy.

Recursive hierarchies:

- ⇒ A recursive hierarchy occurs when a dimension has a self-referential relationship. Means a dimension can be both a parent and a child to itself.
- ⇒ Aggregating data in recursive hierarchies require multiple levels of recursion.

Time

1. Transactional vs Snapshot Schemata

⇒ These are two main ways of representing data in data warehouse

Feature	Transactional Schemata	Snapshot Schemata
Focus	Records individual events or transactions	Captures periodic snapshots of data.
Data Granularity	Fine-grained, detailed data	Coarse-grained, summarized data.
Data Structure	Typically a fact table with detailed measures and dimensions	Fact table with measures and dimensions representing a specific point in time.
Data Update	Frequent updates as new transactions occur.	Periodic updates at specific intervals.
Ex:	Bank deposits/withdrawals, sales, transactions,	Monthly sales figures, end-of-day inventory levels

Data update

Frequent updates as new transactions occur.

Periodic updates at specific intervals.

Data analysis

Suitable for detailed analysis of individual transactions and trends over time.

Best for analyzing trends and comparisons between different periods.

Key Considerations

Data volume and storage requirements, query performance, and real-time processing needs.

Data accuracy and consistency, data retention policies, and data freshness.

⇒ Transactional schemata is ideal for capturing a detailed history of events and analyzing individual transactions.

⇒ Snapshot schemata are better suited for summarizing data at specific points in time and analyzing trends over longer periods.

2. Late update:

⇒ When change something after it's already been recorded.

⇒ Refers to the situation where the values of certain metrics or dimensions associated with a specific event are updated after the initial recording of that event.

⇒ This often occurs due to delays in data processing, errors in initial data, or the need for more accurate information.

for example:

- o imagine a data warehouse tracking sales data. Initially, a sale might be recorded with an estimated revenue figure.

- o Later, when the final invoice is processed, the actual revenue figure might be different and the data warehouse needs to be updated to reflect this change.

Key challenges of late updates:

Data Consistency:

- o Ensuring data consistency across different tables and reports can be challenging when late update occurs.

Data Quality:

- o Can impact the overall quality of the data in the data-warehouse.

Data Analysis and Reporting:

- o Late updates can lead to inaccurate or incomplete analysis and reporting.

Addressing Late updates:

To effectively handle late updates, data warehouse designers and administrators can implement strategies such as

Incremental load processes:

- o Regularly updating the data warehouse with the latest information.

Change Data Capture:

- o Identifying and tracking changes

in source systems to ensure timely updates.

Data Quality Checks:

- Implementing robust data quality checks to identify and correct errors.

Versioning:

- Keeping track of different versions of data to allow for historical analysis.

Alerting Mechanisms:

- Setting up alerts to notify relevant stakeholders about late updates and potential data quality issues.

3. Dynamic Hierarchies:

⇒ Dynamic hierarchies are hierarchies that change over time.

⇒ This is important because the organizations often restructure, and these changes need to be reflected in the data warehouse.

Example:

- a Company with a hierarchical structure for its sales organization.

- Region
- District
- Store.

Overtime, the company might restructure, adding new districts or moving stores between districts. A dynamic hierarchy allows the data warehouse to adapt to these changes.

Four ways to analyze data with dynamic hierarchies:

1. Today - for - today
2. Yesterday - for - today
3. Today - for - Yesterday
4. Today - and - Yesterday

How dynamic Hierarchies are implemented?

⇒ Dynamic hierarchies are typically implemented in data warehouse tools like Power BI or Tableau.

With dynamic hierarchy we can do operations:

- Drill Down
- Drill Up

Overlapping Fact Schemata

⇒ Means how different fact tables in a data warehouse can overlap or share common dimensions.

⇒ Different fact tables in a data warehouse represent different sets of facts or measurements.

⇒ Sometimes need to compare data from two different fact tables.
Drill-across queries

⇒ Query that compares measures across different fact tables is called a "drill-across" query.

Comparable fact schemata

⇒ Defining how to determine if two fact tables can be combined for drill-across.

⇒ Two fact tables can be combined if they have at least one dimension in common.

Example

⇒ In real-world scenarios, finding common dimensions can be challenging.

⇒ There are no perfect way to find common dimensions, but they provide some guidelines:

1. Dimensions should have overlapping values to be considered the same.

2. It is hard to determine if dimensions are the same if they come from different systems with different data types.

3. If the dimensions have the same name in different fact tables, that's a good sign they might be the same.

- A well designed data warehouse dimensions should be consistent across different fact tables.
- If dimensions are consistent, it is easy to identify common dimensions.

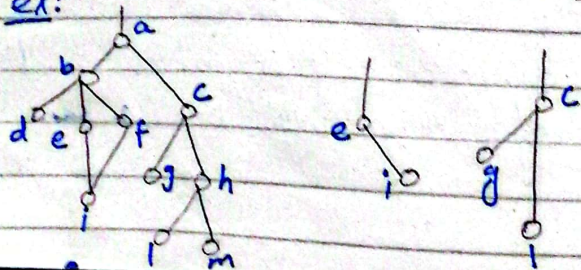
Reducing Hierarchies

- How simplify dimensions when combining fact tables.

Def:

- Taking original tree H and breaking it down into smaller trees. Choose a specific group of relatives (X) and the new smaller trees will only include those relatives and anyone that directly relate to them. (Transitive or functional dependencies)

ex:



Overlapping Compatible fact Schemata

- Two fact tables can be combined if they have at least one dimension in common. This is called "overlapping compatible fact schemata".

- Two compatible fact schemata are called compatible when reduction of their hierarchies to common attributes are equal.

Compatibility:

Fact tables are consistent if they have consistent functional dependencies.

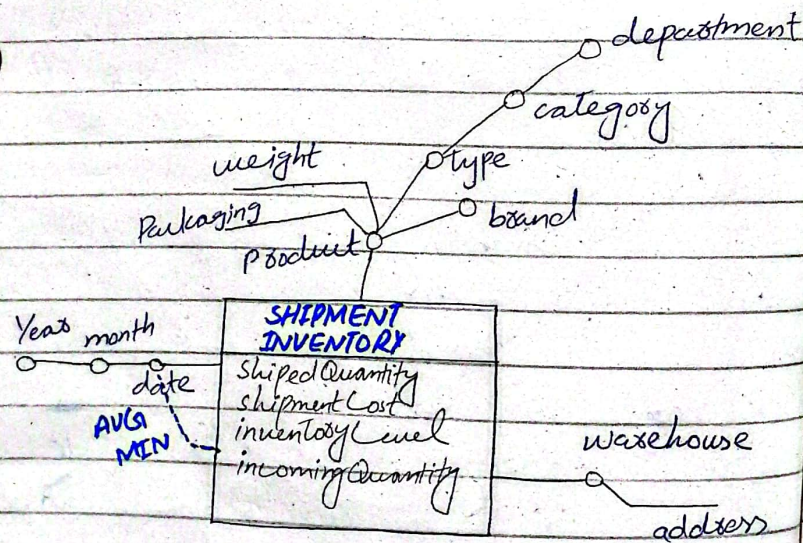
overlapping:

Combining compatible fact tables creates a new fact table with shared dimensions and combined measures.

- Overlapping two compatible fact schemata results in an overlapped fact schema in which:

1. Hierarchies are the common reduction of the hierarchies in source fact schemata.
2. Measures are the union of the measures in source fact schemata.
3. Each attribute domain is an intersection of the corresponding attribute domains in source fact schemata.

EX: Overlapping INVENTORY and SHIPMENT schemata.



Formalizing the Dimensional Fact Model:

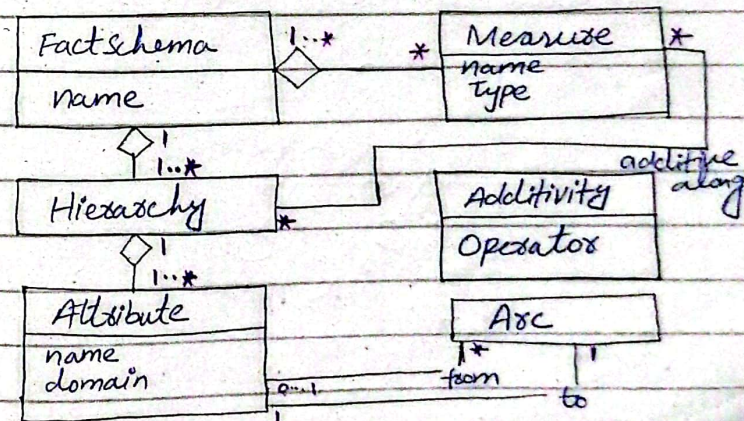
⇒ it defines how to make the Dimensional Fact model more organized and understandable.

⇒ The DFM is a way to arrange data in a multi-dimensional structure, often used in data warehouses.

(1) MetaModel:

o Simplified diagram of DFM. It is a blueprint that shows the different parts of the DFM and how they connect to each other.

ex: UML metamodel of the DFM



(2) intensional properties:

⇒ Properties that define the meaning and relationships between the elements of the DFM.

Hierarchy:

⇒ A hierarchy h is a pair $(A, \leq)$ where

- A is a finite set of dimensional attributes

- $\leq$ is a partial order based on the attributes of A

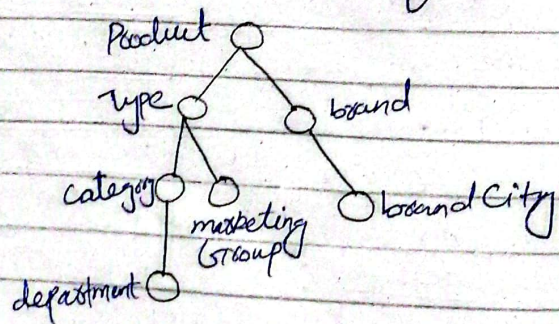
ex:

Product hierarchy

$A = \{ \text{Product, type, category, department, marketingGroup, brand, brandCity} \}$

Hasse diagram of hierarchy:

↳ visual representation of hierarchy.



Fact Schema:

⇒ Let F be a fact schema. And it is defined as a pair (H, M) in which:

- H is a finite set of hierarchies.
- M is a finite set of measures.

Group-by set:

⇒ Let F be a fact schema. a Group by set of F , is a subset of attributes, $G \subseteq \text{Attrs}(F)$ such that for each pair of attributes a_i and a_j contained in G from the same hierarchy h , neither $a_i \leq_n a_j$ nor $a_j \leq_n a_i$ holds.

Roll-up order:

⇒ Given a fact schema F , a partial roll-up order $\leq$ on the set of all possible group by sets of F as follows:

- $G_i \leq G_j$ if and only if $G_i \rightarrow G_j$

(3) Extensional Properties:

⇒ Focuses on the actual data that fills the dimensional Fact model.

Hierarchy instance:

⇒ Given a hierarchy $h = (A, \leq_n)$, an

instance of h is a pair (D, L) where

- D is the set of domains $\text{Dom}(a_i)$ of the attributes in A .
- L is a family of roll-up functions that map each value of $\text{Dom}(a_i)$ for each pair of a_i and a_j attributes such that $a_i \leq a_j$:
$$\text{up}_{a_j}^{a_i} : \text{Dom}(a_i) \rightarrow \text{Dom}(a_j)$$

ex:

$\text{Dom}(\text{city}) = \{\text{'Abbyville'}, \text{'Auburn'}, \dots\}$

$\text{Dom}(\text{state}) = \{\text{'Kansas'}, \text{'Oklahoma'}, \text{'Texas'}, \dots\}$

$\text{up}_{\text{state}}^{\text{city}}(\text{'Abbyville'}) = \text{'Kansas'}$

Coordinate:

- $\text{Dom}(G) = \text{Dom}(a_i) \times \dots \times \text{Dom}(a_j)$

Cube & Primary Event:

- $C : \text{Dom}(\text{Part}(F)) \rightarrow \text{Dom}(m_1) \times \dots \times \text{Dom}(m_n)$

Aggregation and secondary events:

- $C^F : \text{Dom}(G) \rightarrow \text{Dom}(m_1) \times \dots \times \text{Dom}(m_n)$
- $C'(\alpha') \cdot m_i = \bigwedge_{\alpha: \text{up}}^{G(F)} C(\alpha) m_i$